6CS007 Project and Professionalism

Final Year Project

Mid-Year Report

TruthLens: Fake News Detection Platform

Student Name: Rigen Manandhar

Student ID: 2408621

Supervisor: Mrs. Prakriti Regmi

Reader: Mr. Siman Giri

Contents

1 Project Overview

The project addresses the rapid spread of misleading and fabricated information across online news and social platforms. The primary aim is to deliver a practical, accessible system that supports credibility assessment for English-language news content, with emphasis on transparent and interpretable outputs suitable for end users.

The motivation is grounded in the societal and technical challenges of misinformation, where manual factchecking is not scalable and where black-box automated predictions can undermine trust. The project therefore prioritizes a hybrid approach combining machine learning signals, basic source credibility cues, uncertainty handling, and explainability to support informed user judgement.

Objectives:

- Develop an end-to-end platform that supports both text-based and URL-based fake news analysis within a unified user experience.
- Train and integrate transformer-based text classification models for short-text and long-article analysis.
- Provide interpretable outputs through explainable AI techniques and explicit uncertainty signalling when confidence is insufficient or when model signals conflict.
- Validate the system using established classification metrics and robustness-focused evaluation procedures, while documenting limitations and ethical considerations.

Expected Outcomes:

- A working web application that accepts news text and article URLs and returns credibility-oriented outputs.
- A backend API that performs extraction, preprocessing, inference, and explanation generation.
- A hybrid inference design that can route inputs to appropriate models and handle conflicting or low-confidence results.
- A Chrome extension prototype enabling in-context analysis within a browsing workflow.

- A documented evaluation framework with placeholders for final quantitative results, plots, and error analysis artifacts.

2 Work Completed So Far

At the mid-year stage, the project has progressed through literature analysis, requirements definition, system design, and the implementation of a functioning prototype spanning the model layer, backend services, web frontend, and a Chrome extension. Testing activities have begun, with emphasis on integration correctness and early model sanity checks. Comprehensive quantitative evaluation is scheduled for the remaining project period and is described with appropriate placeholders.

2.1 Literature Review / Background Research

The background research reviewed contemporary approaches to fake news detection, including classical machine learning pipelines based on hand-crafted linguistic features, as well as modern transformer-based architectures such as BERT and RoBERTa. The literature indicates that transformer models dominate text-based detection due to their capacity to model contextual semantics. However, recurring limitations were identified: reliance on dataset-specific shortcuts, weak cross-domain generalization, and limited end-user interpretability.

The review also considered multimodal detection methods that fuse textual and visual cues, but these were assessed as computationally expensive for a low-latency web system at the current project scope. As a result, a text-first approach was selected for the initial deliverable, with a clear pathway for future multimodal expansion.

Key conclusions informing project direction:

- Deployment constraints strongly motivate a text-first design, prioritizing stable inference latency and manageable resource consumption.
- Dataset artifacts and leakage risks necessitate careful preprocessing, deduplication, and split strategies to reduce shortcut learning.
- Explainability is essential for user trust; interpretable features and post-hoc explanations should be integrated as first-class system outputs.

Summary Table of Reviewed Papers and Design Implications:

Paper	**Method**	**Dataset(s)**	**Key Results**	**Limitations**	**How it informs this project**
Suryavardan et al. (2023) – Factify 2	Multimodal entailment baseline (ViT + Sentence-BERT).	FACTIFY 2 (text + images; includes satire).	Baseline Macro F1-score reported as 65%; multimodal fusion remains challenging.	Sensitive to image quality/relevance; increased complexity for real-time deployment.	Supports a text-first deployable approach at mid-year stage; multimodal detection is positioned as future work.
Singhal et al. (2021) – Inter-modality discordance	Multi-task (headline + body) with multiple images; contrastive loss for discordance.	FakeNewsNet	Reported F1-score of 91.5%; discordance modelling improves detection.	Sensitive to noisy images; computationally costly.	Motivates explicit handling of mixed signals and careful trade-offs for real-time performance.
Giachanou et al. (2020) –	BERT + VGG-16 + LSTM; cross-modal similarity features.	GossipCop	Reported F1-score of	Fusion complexity: tags may	Reinforces strong text baseline

multi-image detection			79.85%; multi-image similarity can help vs text-only baselines.	lose semantic nuance; computational cost.	now and a roadmap for multimodal extension later.
Zhang et al. (2024) – Image–text similarity	Analysis of multiple similarity measures; includes ViLBERT-based features.	Not specified in the review summary.	Finds fake news can have high image–text similarity; alignment alone is insufficient.	Computationally heavy similarity pipelines; limited real-time suitability.	Strengthens emphasis on robust text verification and explainability; informs future multimodal design beyond naive similarity.
Pavlov & Mirceva (2022) – Domain-specific transformers	Fine-tuned BERT and RoBERTa on COVID-19 posts.	COVID-19 social media dataset	RoBERTa reported to outperform BERT; domain adaptation improves	Domain-specific; may not generalize to formal news articles.	Supports selecting RoBERTa and motivates curated/updated data collection

			outcomes.		for future experiments.
Abedalla et al. (2021) – Deep learning perspective	Stance detection with CNNs/Bi-LSTMs + attention.	FNC-1	Highlights attention for interpretability and warns about leakage in evaluation.	Text-only; not presented as a deployable user-facing system.	Informs explainability expectations and evaluation rigor (leakage control and valid splits).
Gautam et al. (2022) – NepBERTa	Language-specific transformer pre-training for Nepali.	Large Nepali corpus	Shows language-specific pre-training can outperform generic multilingual models.	Language-specific focus; multimodality not addressed.	Supports strong English baseline first while outlining future multilingual pathway.
Safdar & Wasim (2024) – RAFAH	RoBERTa + hybrid engineered features (emotional/linguistic/psycholinguistic).	BuzzFeed; ISOT	Reports state-of-the-art performance; hybrid features can	Feature engineering adds complexity and development effort.	Motivates hybrid signals and explainability to strengthen user trust

			improve transparency.		while balancing real-time feasibility.
Jain & Kasbe (2018) – Naive Bayes + scraping	Naive Bayes with dynamic scraping to refresh data.	Scraped articles	Reports AUC of 0.931; full-text analysis can outperform headline-only approaches.	Bag-of-words limits semantics; limited real-time integration.	Reinforces URL scraping and full-article support as core platform features.
Mridha et al. (2021) – Comprehensive review	Survey of DL methods and open challenges.	Survey (multiple datasets across studies)	Highlights transformer dominance, hybrid features, explainability, and robustness as key themes.	Survey-level guidance; limited deployment detail.	Provides a roadmap aligning with RoBERTa + explainability + robustness in a user-facing web platform.

2.2 Requirements Gathering & Analysis

Requirements were derived from the project objectives and from usability considerations for real-world verification workflows. The requirements emphasize ease of input, clarity of outputs, and responsible presentation of uncertainty. Functional requirements focus on text and URL analysis, explanation capability, and basic result traceability. Non-functional requirements focus on reliability, latency, privacy, and maintainability.

Functional requirements (defined and partially implemented):

- Accept input as pasted text (headline, excerpt, or full article) with input-mode routing where applicable.
- Accept URL input and attempt automated extraction of article text for analysis; return a safe fallback when extraction fails.
- Return a clear verdict category and a confidence-oriented explanation suitable for non-expert users.
- Provide explainability outputs on demand or for uncertain cases and expose a concise analysis trace of applied checks.
- Support a browser extension workflow that submits text and URL context for analysis without requiring manual navigation to a separate tool.

Non-functional requirements:

- Low-latency responses suitable for interactive use, with graceful degradation under extraction or model failures.
- Secure URL handling to avoid unsafe fetching patterns and to reduce exposure to malicious inputs.
- Privacy-aware processing avoids unnecessary retention of user-submitted content and clearly communicates limitations.
- Maintainability through modular separation between frontend, backend, extraction, model inference, and explainability logic.

Functional requirement (FR) and Non-Functional Requirement (NFR)

Req ID	Requirement	Type	Priority	Implementation status
FR-01	Allow users to submit pasted news text (headline/excerpt/full text) for analysis.	Functional	High	Completed
FR-02	Allow users to submit news article URL and attempt automatic extraction of article text for analysis.	Functional	High	Completed
FR-03	Support input-mode selection (auto/headline only/full article/headline + article) to control routing and analysis behavior.	Functional	Medium	Completed
FR-04	Return a clear verdict category and risk level suitable for end users (e.g., suspicious/likely real/uncertain).	Functional	High	Completed
FR-05	Provide confidence-oriented signals in the result (e.g., per-model confidence and/or clear uncertainty messaging when confidence is low).	Functional	High	Completed
FR-06	Provide an analysis trace showing key checks performed (e.g., parsing/routing, source check, headline check, article check).	Functional	Medium	Completed
FR-07	Provide explainability output for uncertain cases and/or when the user requests an explanation (e.g., LIME feature contributions).	Functional	High	In progress
FR-08	Provide a fallback response when URL extraction fails (timeout/paywall/no content), including a clear reason message.	Functional	High	Completed
FR-09	Provide basic source credibility scoring using a small, curated domain list (e.g., REAL/BIASED/SATIRE/FAKE) as a supporting signal.	Functional	Medium	Completed
FR-10	Provide a working Chrome extension prototype that can submit text/URL context for analysis using the same API contract as the web application.	Functional	Medium	Completed
FR-11	Store and retrieve user preferences related to detection settings (e.g., preferred input mode, explanation mode) where applicable.	Functional	Medium	Completed
FR-12	Provide a news browsing component (headline feed + search/filter) to support testing and demonstration of real-world content.	Functional	Low	Completed
NFR-01	Maintain interactive response time for typical text inputs; degrade gracefully for long inputs and URL extraction.	Non-functional	High	In progress
NFR-02	Ensure safe URL fetching to reduce security	Non-	High	Completed

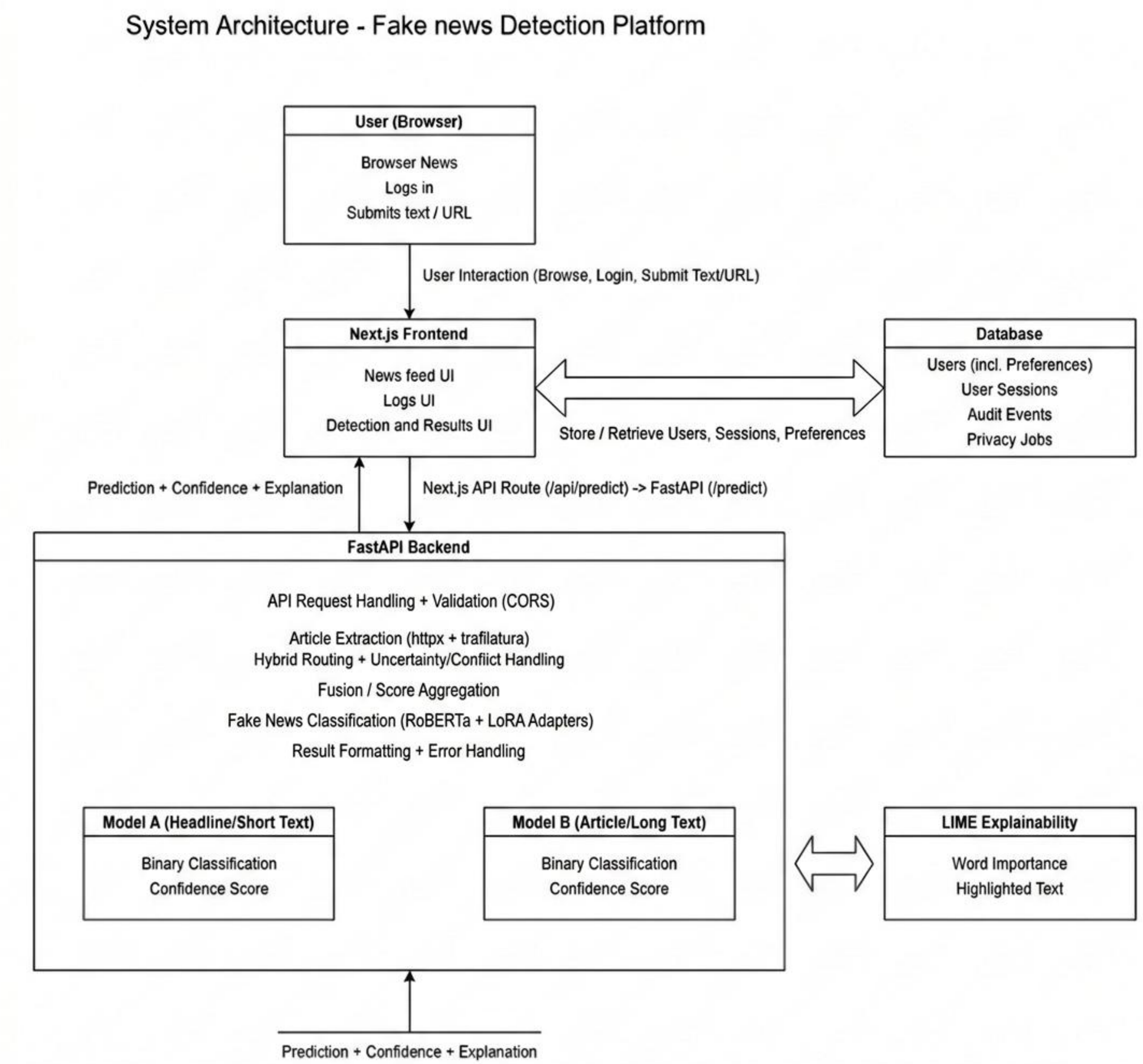
	risks (block localhost/private IPs; enforce timeouts; limit redirects).	functional		
NFR-03	Reliability: handle invalid inputs and backend failures with clear, user-friendly error responses.	Non-functional	High	Completed
NFR-04	Maintainability: keep model inference, extraction, explainability, and API logic modular to support future updates.	Non-functional	Medium	Completed
NFR-05	Privacy: minimize retention of user-submitted text/URLs; document how data is handled and what is stored.	Non-functional	Medium	Planned
NFR-06	Robustness: reduce shortcut learning/leakage via preprocessing, deduplication, and stronger evaluation splits; report limitations transparently.	Non-functional	High	In progress
NFR-07	Usability: present results with clear language, progressive disclosure (summary first, details on demand), and accessible UI.	Non-functional	Medium	In progress

2.3 Design

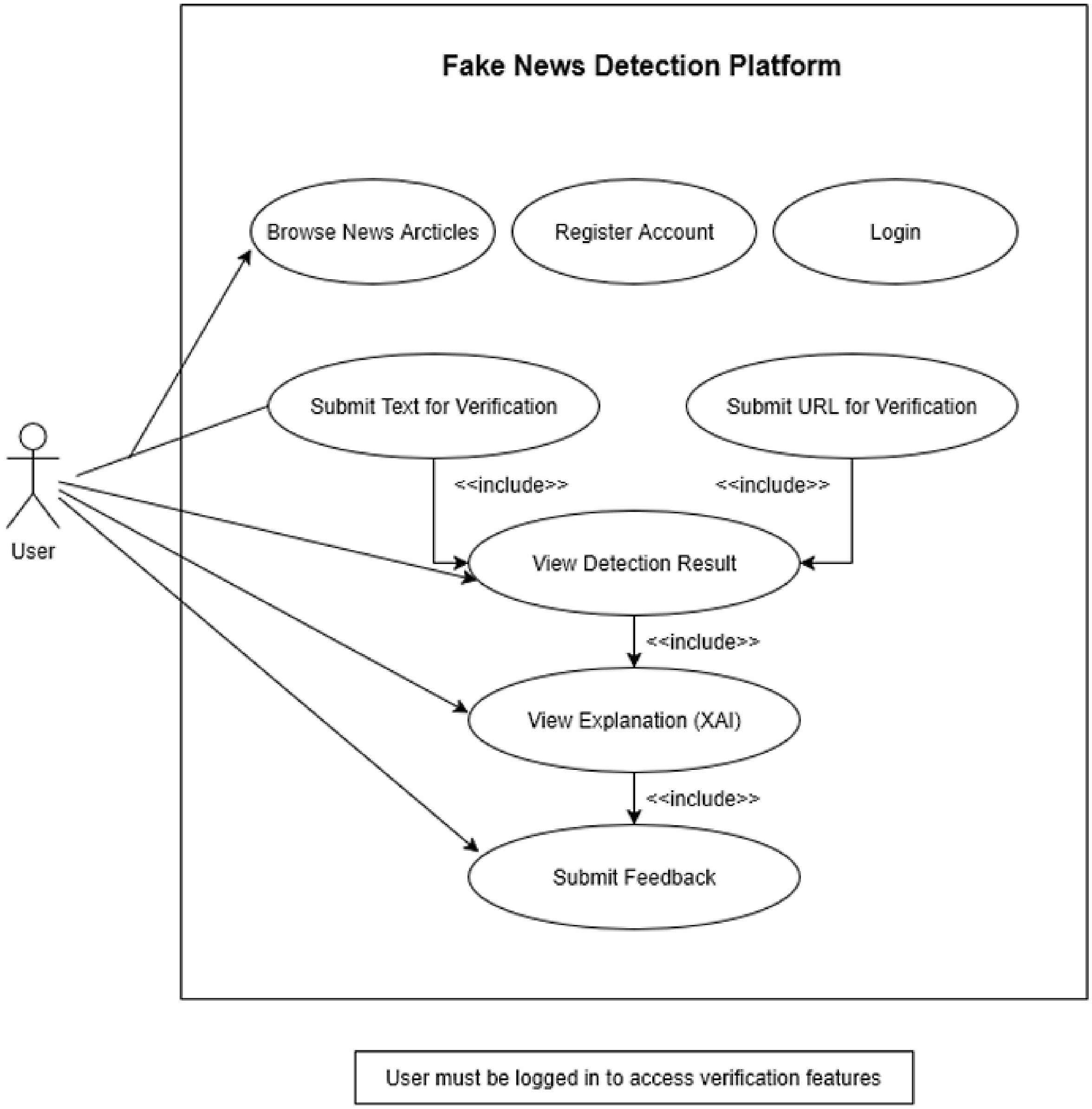
The system design follows modular architecture, separating user interaction layers from backend services and model inference. A hybrid design is adopted to route short inputs and long-form articles to appropriate models, while combining these signals with basic source credibility cues and explicit uncertainty handling. Design artifacts were produced to describe behavior, data flow, and storage needs.

Design artifacts prepared:

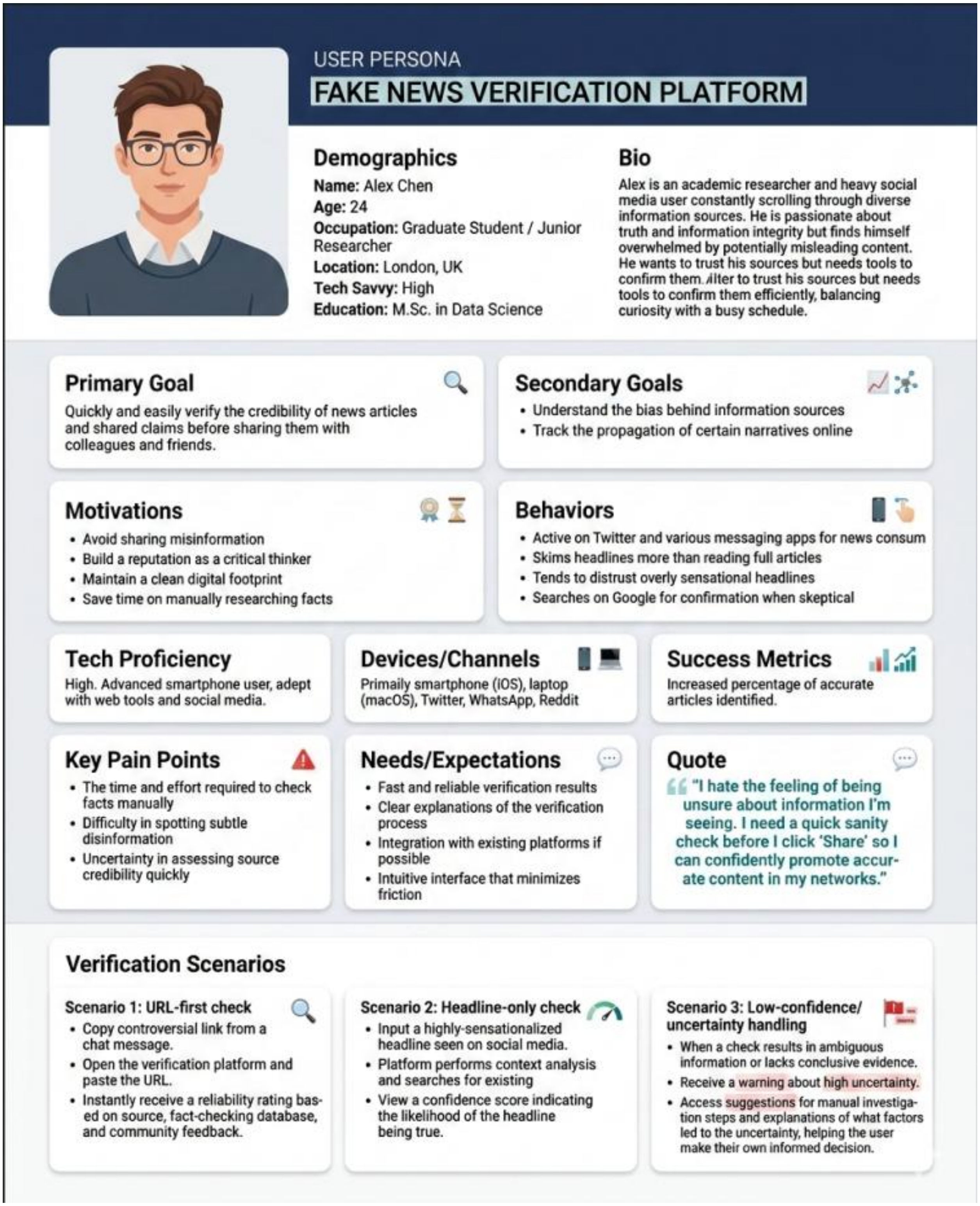
System Architecture Diagram:



Use Case Diagram:



User Persona 1:



User Persona 2:

FACT-CHECKING VOLUNTEER

FAKE NEWS VERIFICATION PLATFORM

Name: Alex Jenkins
Photo/Avatar:
Role/Context: Fact-Checking Volunteer & Journalism Student
Demographics:
Age: 26
Location: Chicago, USA
Education: M.Sc. Candidate in Journalism & Digital Media
Experience: 1.5 Years in Citizen Journalism

Primary Goal

Verify claims accurately and defend conclusions with documented evidence.

Workflow

1. Triage flagged content.
2. Initial source check.
3. Cross-reference claims with reliable databases and outlets.
4. Coliect and organize evidence.
5. Write and submit fact-check reports.
6. Review feedback.

Tools Used

Laptop
Web Browsers, News Aggregators, Social Media (Twitter, Reddit), Academic Databosses (JSTOR), Search Engines, Collaborative Docs, Existing Checking Sites (PolitiFact, Snopes)

Bio

A passionate advocate for truth in media, Alex balances her studies with volunteer fact-checking for a grassroots non-profit. She is analytical, persistent, and thrives on methodical research.

Alex uses digital platforms extensively for cross-referencing and seeks reliable, verifiable sources to build strong, defensible cases. She is highly tech-literate but finds current tools fragmented.

Motivations

- Champion truth and media literacy.
- Combat disinformation campaigns.
- Build a robust portfolio.
- Improve journalism skills.
- Gain recognition in the field.

Behaviors

- Cross-checks everything.
- Dives deep into source credibility.
- Uses multiple browser tabs.
- Saves links and screenshots.
- Cites sources meticulously.

Key Pain Points

- Fragmented and unreliable tools.
- Time-consuming manual verification.
- Lack of transparency in existing detection.
- Inconsistent evidence formats.
- Burnout from high volume.

Needs/Expectations

- Fast initial analysis and filtering.
- Clear explanation and traceability of automated detection.
- Easy evidence gathering and formatting.
- Collaboration features.
- A reliable audit trail.

> *"Facts are the foundation of trust. My mission is to ensure every fact is solid, verifiable, and transparently explained, so truth prevails in the digital age."*

Decision Criteria

- Explainability/Transparency.
- Defensibility of decisions.
- Accuracy and speed.
- Integration with research workflow.
- Source diversity.

Success Metrics

- Number of claims verified per week.
- Quality and thoroughness of reports.
- Acceptance rate of submitted fact-checks.
- Reduction in manual research time.
- Accuracy of preliminary automated assessments.

Verification Scenarios (3)

1. Long-Form URL Verification

Receive a suspicious, lengthy URL from a obscure site sharing wild claims.
Cross-reference domain registration, analyze site history, and compare content to known narratives.
Use platform tools for source analysis and historical data.

2. Headline vs. Article Mismatch/Conflict

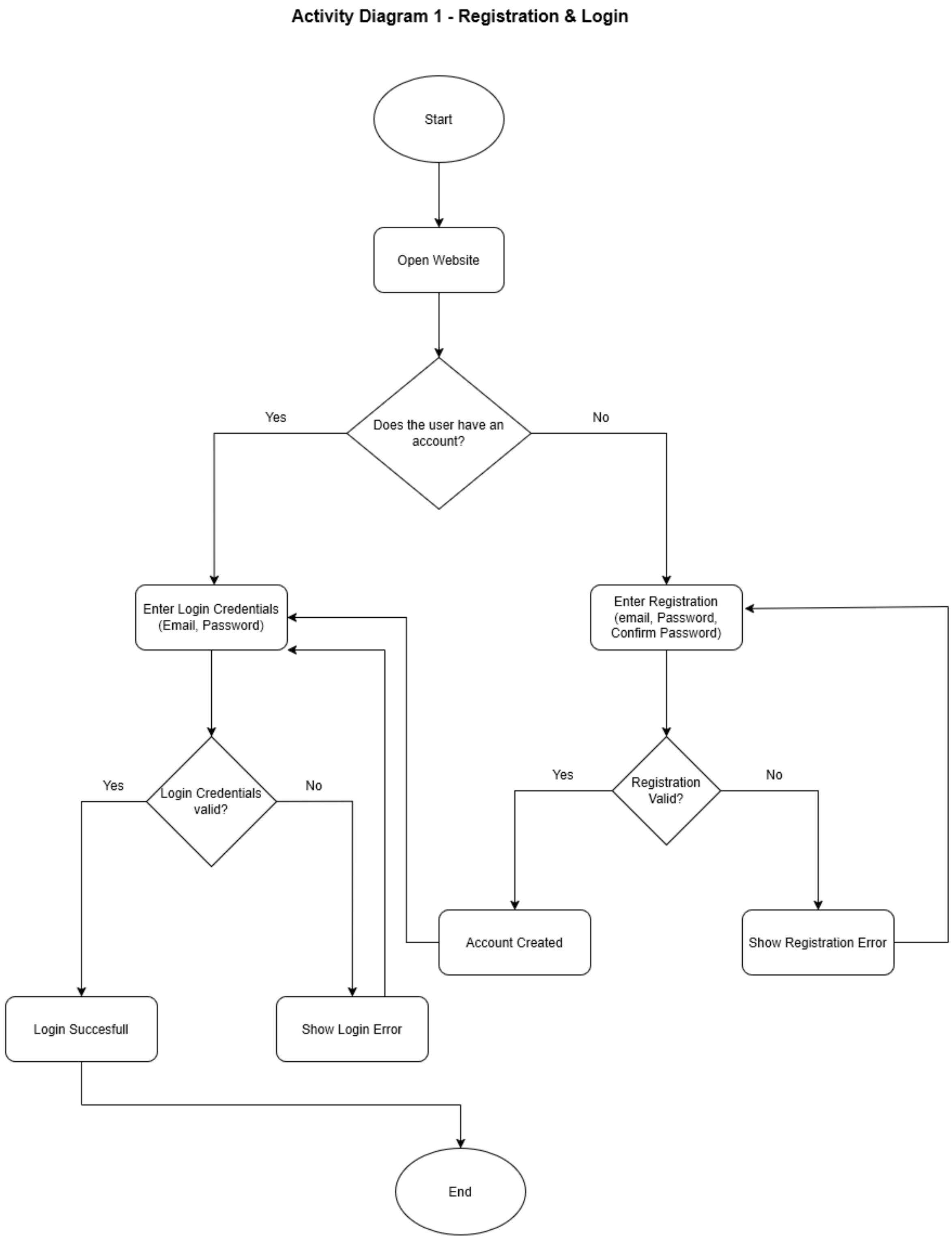
Evaluate a shared social media post where the bold headline directly contradicts the content of the linked article.
Flag and document the discrepancy, highlighting the misleading nature.
The platform aids in quick headline-text comparison and context detection.

3. Extraction Failure/Paywall Fallback to Manual Paste

Encounter a relevant paywalled article or a site where automated content extraction fails.
Seamlessly switch to manually pasting key claims, names, or quotes into the verification tools to continue the fact-checking process.

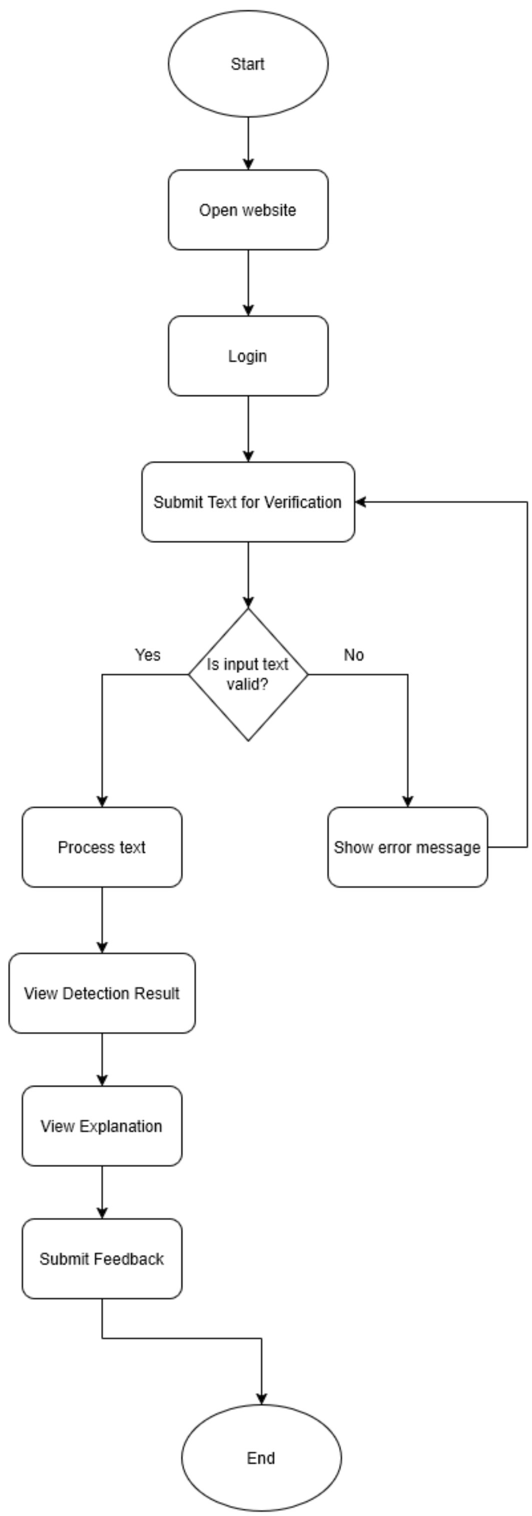
Activity Diagrams:

1. **User Registration and login**



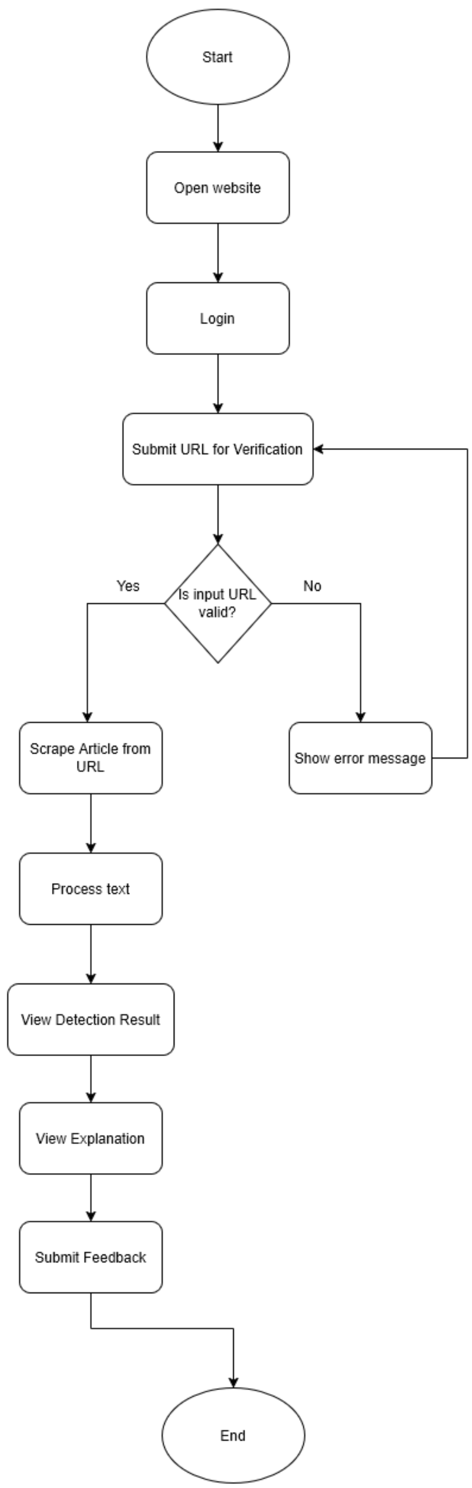
2. Text-Based Fake News Detection

Activity Diagram 2 - Text-Based Fake News Detection



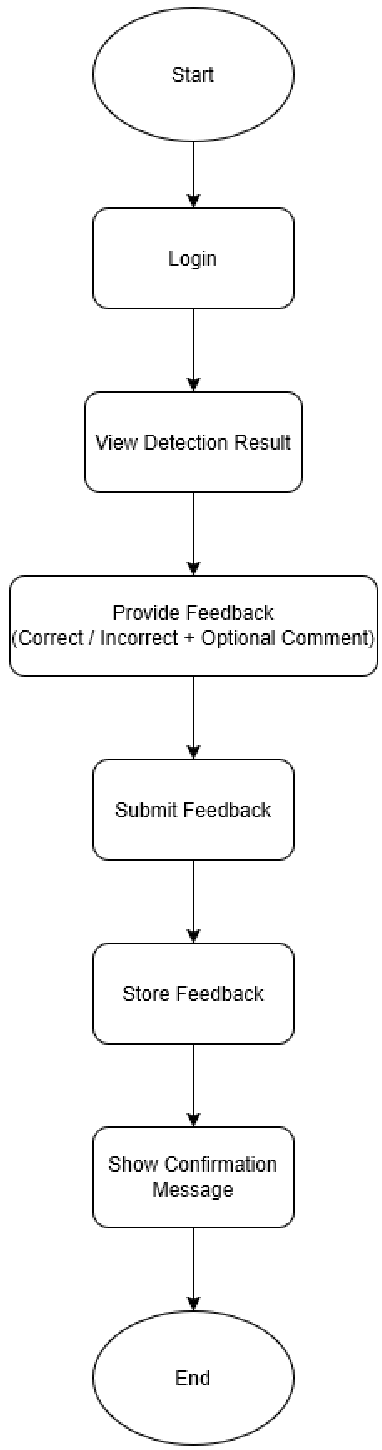
3. URL-Based Fake News Detection

Activity Diagram 3 - URL-Based Fake News Detection



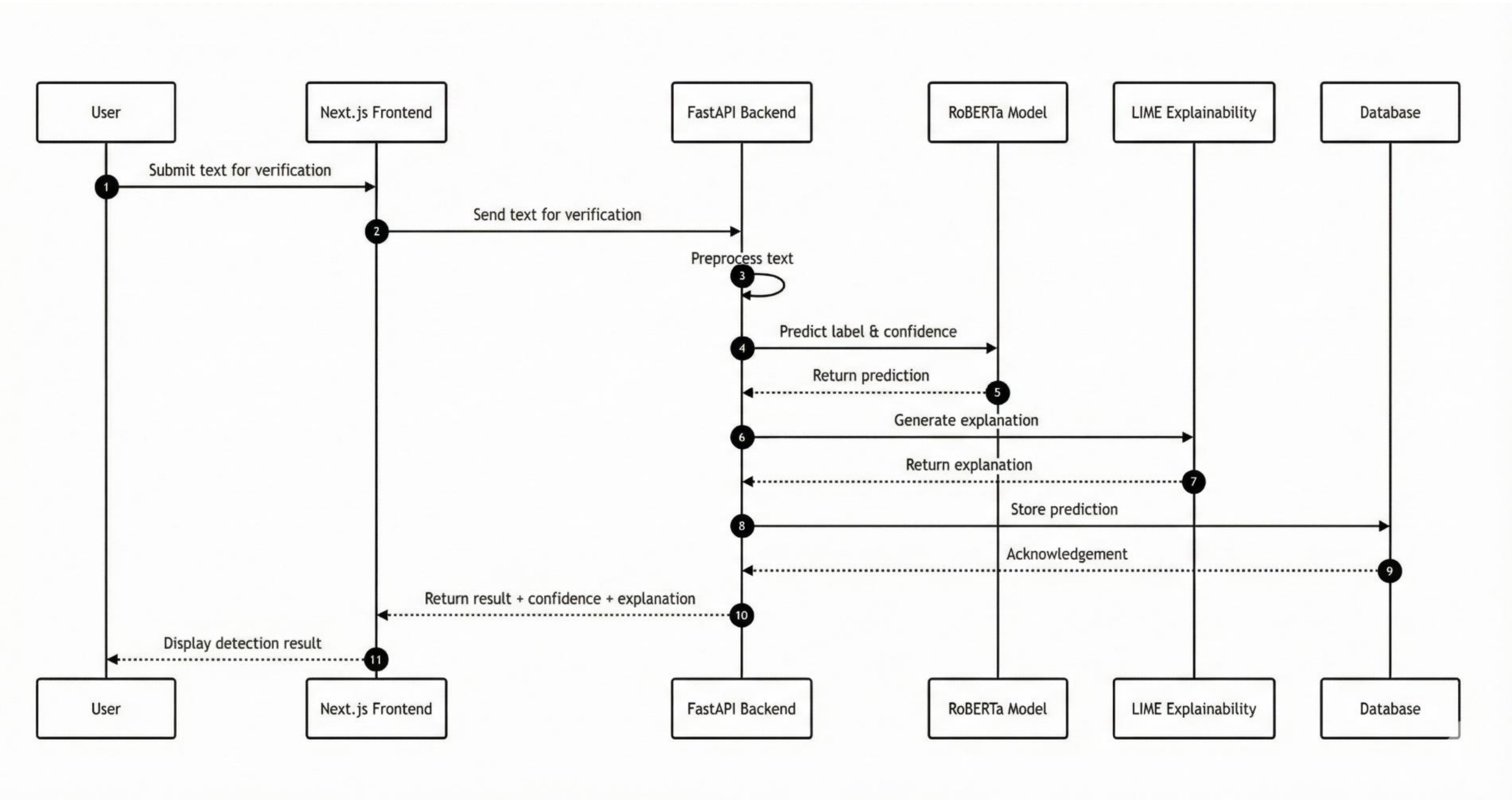
4. Feedback Submission

Activity Diagram 4 - Feedback Submission

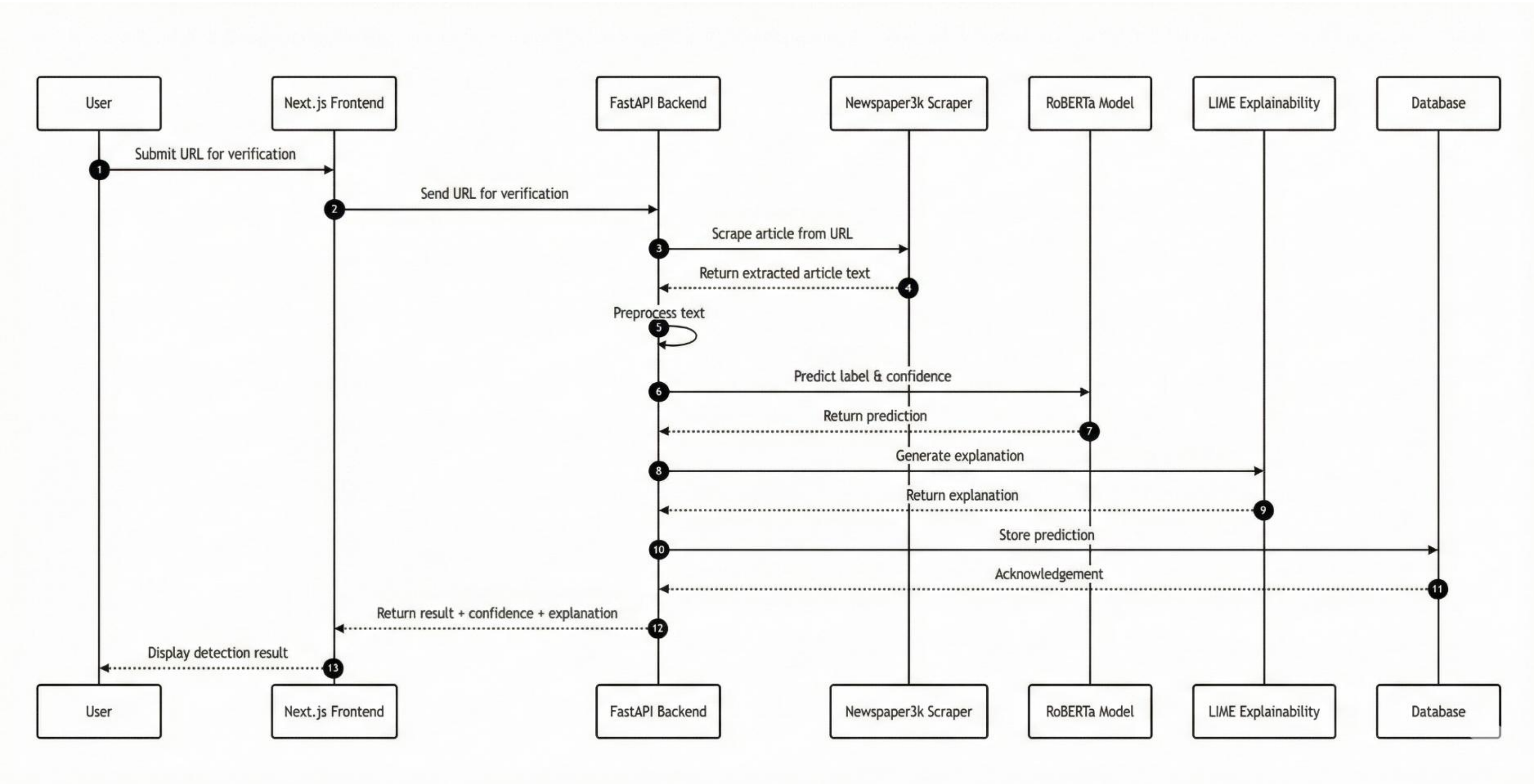


Sequence Diagrams:

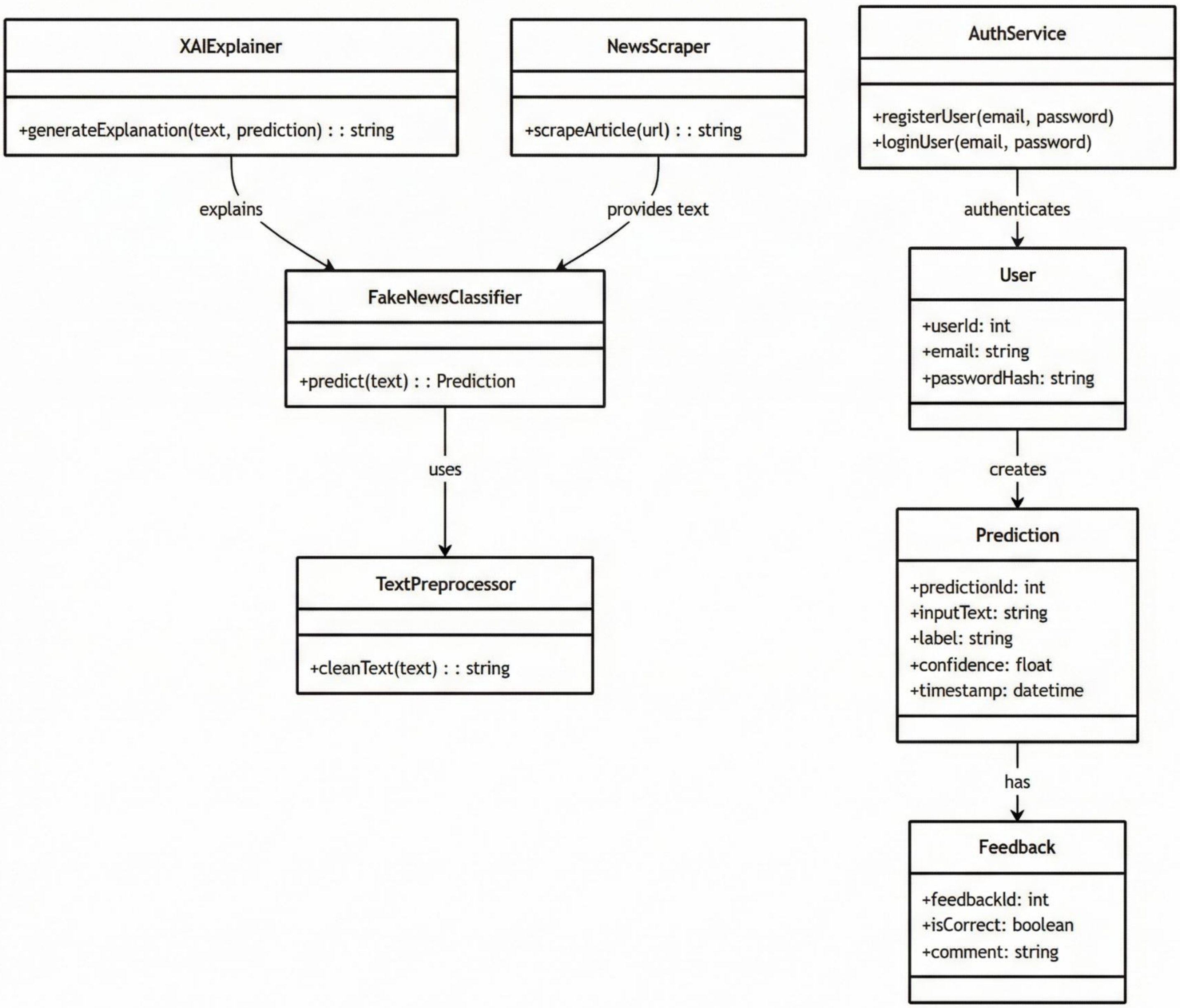
1. **Text-Based Fake News Verification**



2. **URL-Based Fake News Verification**

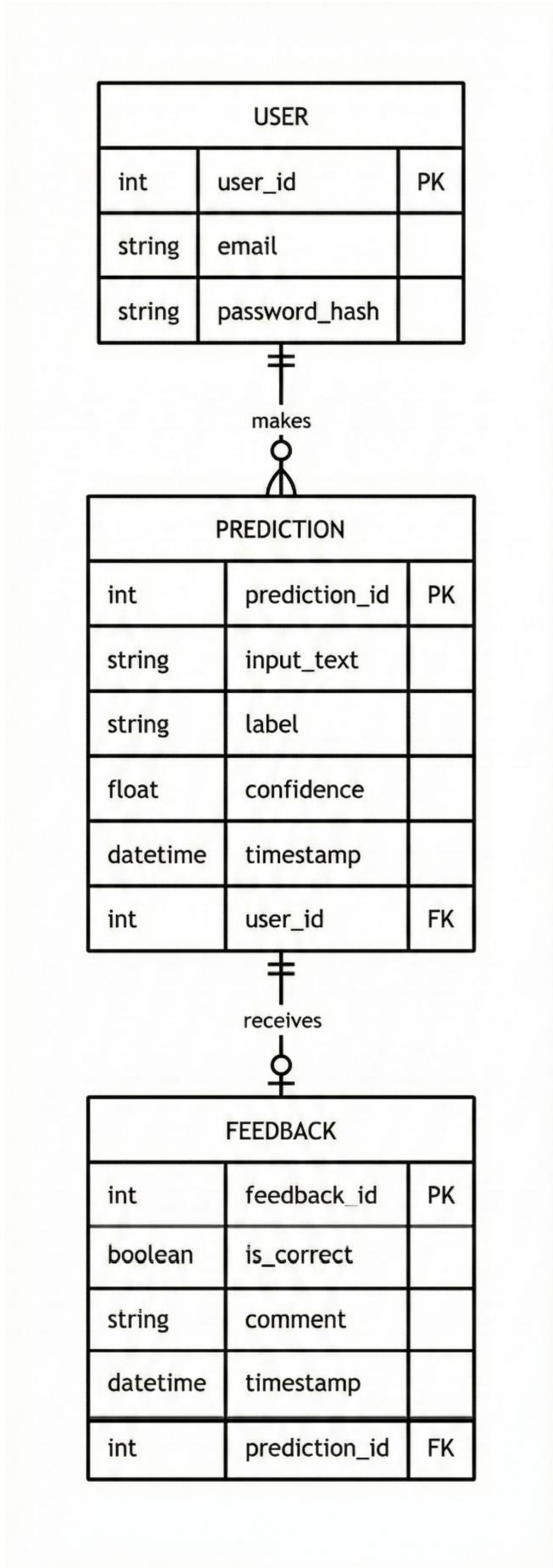


Class Diagram:



XAIExplainer
+generateExplanation(text, prediction) : : string
NewsScraper
+scrapeArticle(url) : : string
AuthService
+registerUser(email, password)
+loginUser(email, password)
explains
provides text
authenticates
FakeNewsClassifier
+predict(text) : : Prediction
User
+userId: int
+email: string
+passwordHash: string
uses
creates
TextPreprocessor
+cleanText(text) : : string
Prediction
+predictionId: int
+inputText: string
+label: string
+confidence: float
+timestamp: datetime
has
Feedback
+feedbackId: int
+isCorrect: boolean
+comment: string

Entity Relationship (ER) Diagram:



Wireframe:

1. **Landing Page**

2. Fake News Detection Page

3. Login Page:

4. Signup Page

2.4 Implementation

- Set up development environment and version control (GitHub/GitLab)
    - A dedicated repository was created with separate modules for the web application, backend API, and browser extension. Environment configuration was externalised using .env variables, enabling reproducible local setup and consistent deployment configuration.
- Established project structure and coding standards
    - A modular structure was adopted to separate concerns (UI, API routes, data access, ML inference, extraction, and explainability). Type safety and validation were enforced via typed interfaces on the frontend and schema validation on the backend. Linting was configured for the frontend to maintain code quality and consistency.
- Implemented core features:
    - Feature 1: Hybrid fake news prediction API (what works)
        - A POST /predict inference endpoint accepts either raw text (headline/claim) or a URL. For URL submissions, article text extraction is

attempted, after which the hybrid pipeline routes inputs to a short-text model (Model A) and/or long-text model (Model B). The API returns a structured output containing the verdict, risk level, a step-by-step trace of applied checks, and explicit uncertainty/conflict handling.

  - Feature 2: End-to-end platform integration (what works)
    - A Next.js web interface supports submission (text/URL), result visualization (verdict + confidence), and on-demand explainability (LIME-based token importance). A browser extension prototype supports quick verification from the browser by capturing the current URL and/or selected text and displaying the returned verdict in the extension UI.

- Database connection and basic CRUD operations
  - MongoDB integration was implemented for user and settings persistence, including user preferences updates and session-related data storage. Core collections were prepared for operational logging (audit events) and privacy-related actions (privacy jobs), supporting account-level management and platform governance features.

2.5 Testing

Testing has commenced with emphasis on correctness of the end-to-end pipeline and reliability of extraction and inference under typical user inputs. The current stage focuses on smoke testing, integration testing across frontend and backend boundaries, and early model sanity checks. A structured evaluation plan has been established for the final phase, including metric reporting and robustness checks, but final numeric results are deferred.

Testing performed to date:

- Manual end-to-end testing for text-only and URL-only inputs, including error cases such as invalid URLs and extraction failures.
- Integration testing of API proxying between the web frontend and the backend inference service.
- Basic verification of explanation generation behavior for uncertain outcomes and for user-triggered explainability.

- Initial checks for stability under longer inputs, including truncation behavior and routing correctness.

3 Technology Stack / Tools Used

The technology stack was selected to support rapid iteration, reproducible experimentation, and reliable full-stack integration. The system combines modern web development frameworks with established machine learning tooling for transformer-based text classification and explainability.

Core technologies:

- Frontend: Next.js (React) with TypeScript for a structured, component-driven user interface and API routing.
- Backend: FastAPI (Python) for an explicit, type-driven API layer and integration of extraction, inference, and explainability.
- Modeling: PyTorch with Hugging Face Transformers and parameter-efficient fine-tuning techniques for transformer adaptation.
- Explainability: LIME for post-hoc, human-interpretable feature attributions over input text.
- Database: MongoDB for persistence of user accounts and configurable preferences where applicable.
- Extension: Chrome Extension (Manifest V3) for in-context analysis from the browser.

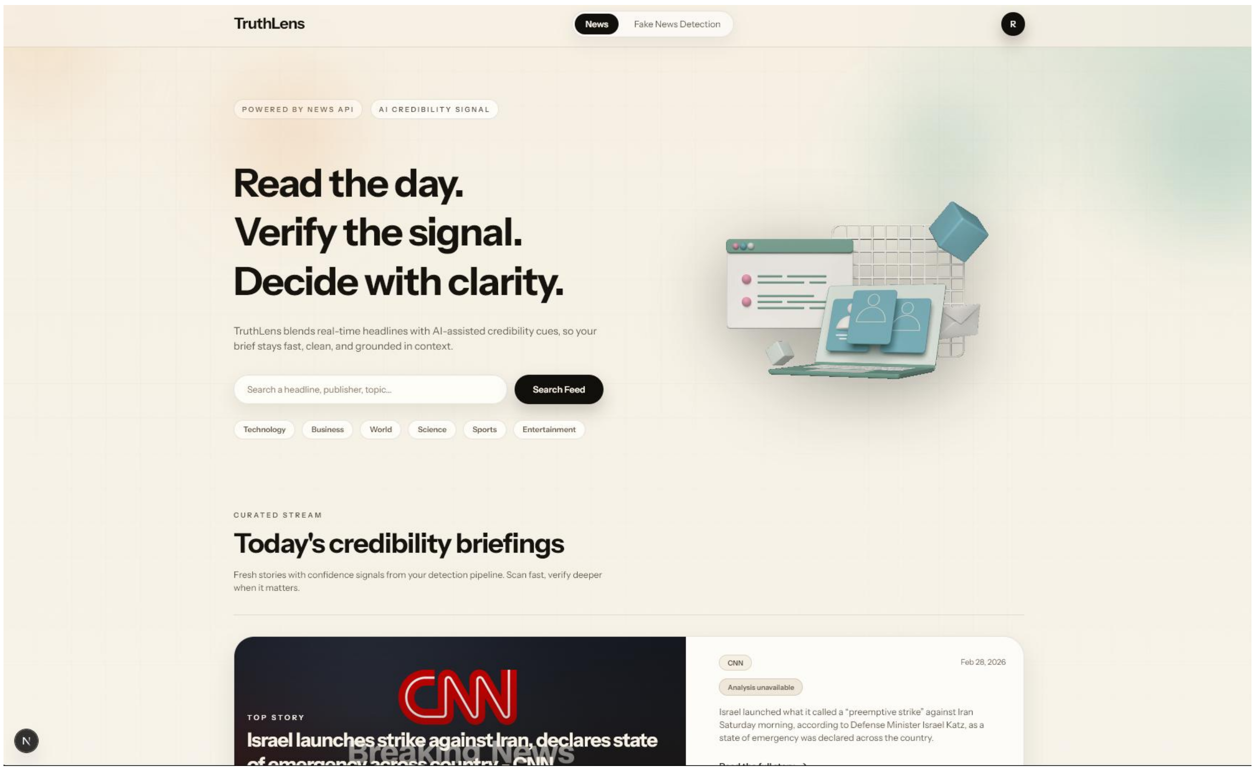
Development and evaluation tools:

- Version control: Git with a remote repository workflow for collaboration and change tracking.
- Development environment: Visual Studio Code and common API testing tools for iterative debugging and validation.
- Data processing: Python-based preprocessing and dataset handling utilities.
- Evaluation: Standard classification metrics and reporting utilities for final quantitative results and plots.
- Design Tools: Figma was used mainly for mock designs and wireframes for the website.

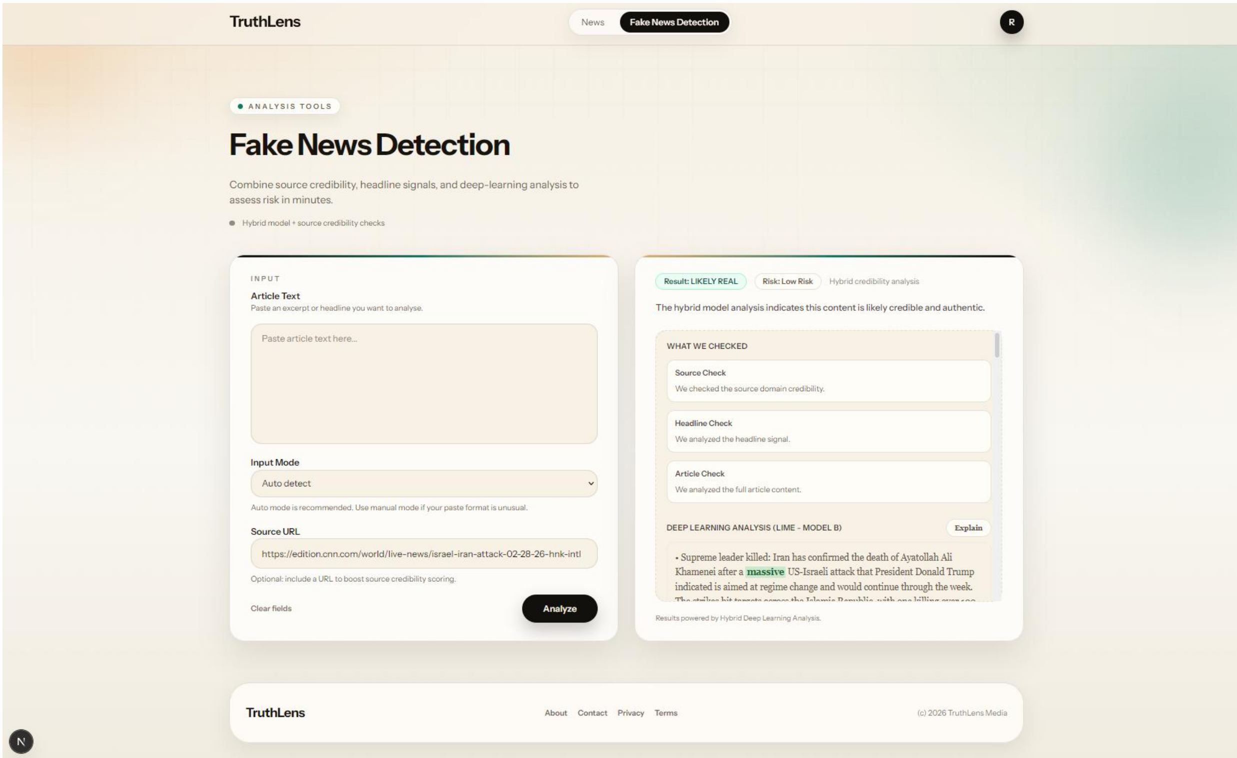
4 Progress Showcase

This section summarises demonstrable progress artifacts from the prototype. The final submission will include screenshots, sample outputs, and representative user flows that validate the system's integration and usability.

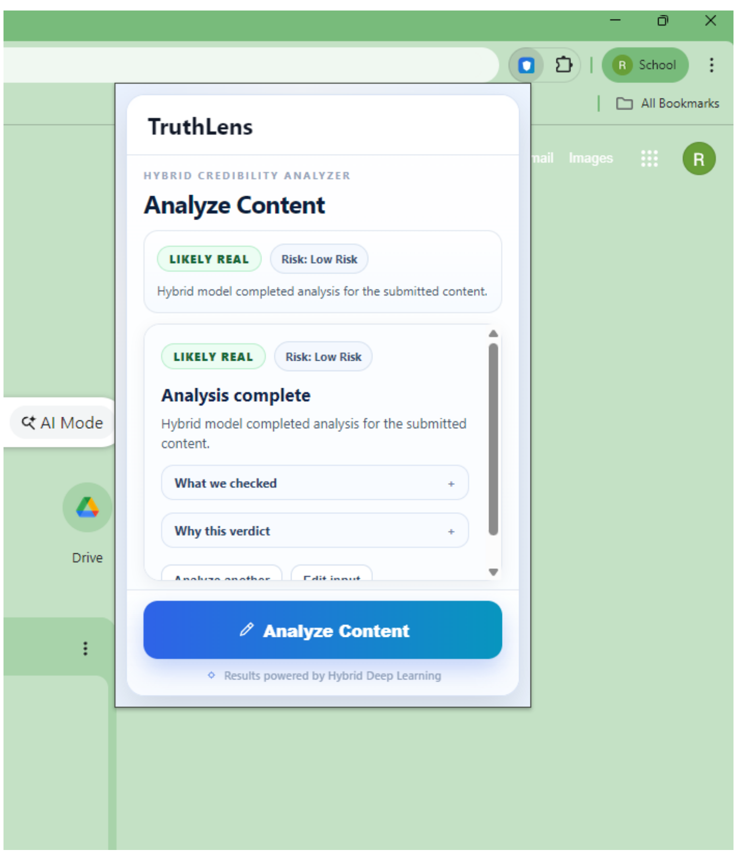
Home Page:



Fake News Detection Page:



Chrome Extension:



5 Challenges Faced

Several technical and project-management challenges have arisen during the mid-year period. The project mitigations emphasize robust engineering practices, careful dataset handling, and incremental integration to reduce end-stage risk.

Key challenges and mitigation actions:

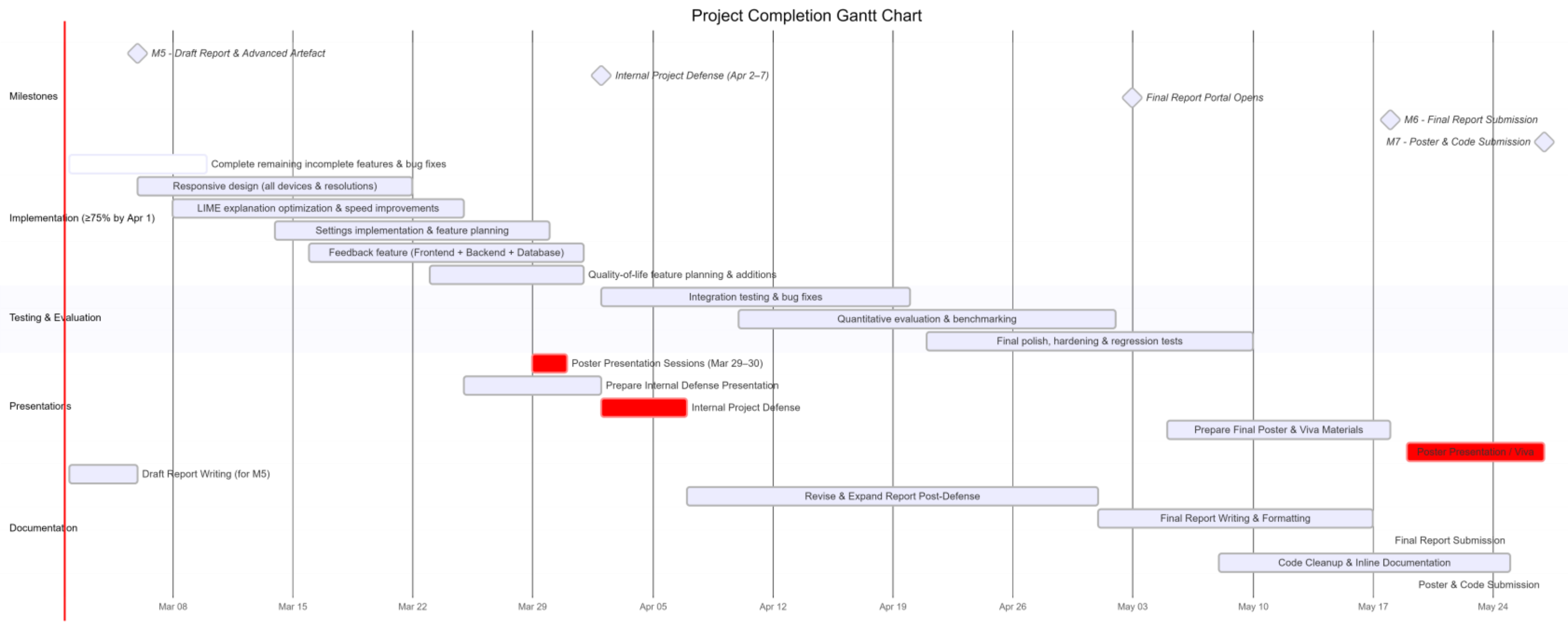
- Dataset shortcuts and leakage: mitigated through preprocessing, deduplication, and stronger split strategies to reduce spurious correlations.
- Model overconfidence and domain shift: addressed via uncertainty thresholds, conflict handling between model signals, and planned calibration analysis.
- URL extraction failures (paywalls, dynamic sites, access restrictions): handled with safe timeouts, clear user messaging, and fallback analysis behaviors.
- Latency constraints for interactive use: mitigated by routing logic and by prioritizing efficient inference and explanation generation only when required.
- Cross-component integration complexity: reduced through a stable API contract and incremental end-to-end testing.

6 Remaining Work

The remaining work focuses on hardening the prototype into a robust demonstrator, completing quantitative evaluation, and producing final deliverables and documentation. The tasks are grouped into implementation completion, evaluation, and submission preparation.

- Making the website more responsive and friendly to all device and resolution sizes.
- Adding feedback features in the front end as well as backend and database.
- To make the LIME explanation more optimized and reduce analyzing time.
- Implementing settings and planning out the settings features more seriously
- Plan out if more features need to be added for quality-of-life improvements.
- Finish off areas that still have no functional weight.

7 Timeline for Completion (Gantt Chart)



8 Risk Assessment and Mitigation (Optional)

The project risks below summarize the most significant technical and delivery threats identified at mid-year and the mitigation strategies planned. The risk register will be revised as evaluation results and integration work progresses.

Risk	**Impact (H/M/L)**	**Mitigation Strategy**
Dataset shift and weak generalization	High	Use source-disjoint and cross-dataset checks; expand robustness testing; tune uncertainty thresholds.
Extraction failures for real-world URLs	High	Improve fallback handling; add clearer user messaging; validate on diverse sites and formats.
Model overconfidence and	High	Apply calibration analysis;

misleading certainty		enforce low-confidence uncertainty; add conflict handling.
Integration complexity across components	Medium	Maintain stable API contract; incremental integration tests; modular interfaces.
Technical debt accumulation	Medium	Scheduled refactoring; code reviews; enforce consistent style and validation checks.
Schedule compression near submission	Medium	Front-load evaluation and integration; maintain weekly milestones; reduce scope if required.

9 Supervisor Meetings Summary

Supervisor engagement has been maintained throughout the mid-year period to ensure alignment with objectives, scope, and evaluation expectations. Meetings were used to review progress artifacts, identify risks, and prioritise the remaining work for the final phase.

- Number of meetings attended: 9
- Number of feedback received: 3
- Number of Actions taken based on feedback: 3

10 Self-Assessment

Progress: On Track. The project has completed the foundational phases of research, requirements definition, and system design, and a working prototype has been implemented across the model layer, backend services, web frontend, and extension workflow. The remaining work is substantial but achievable within the semester timeframe, with a focused plan for evaluation, robustness improvements, and documentation.

Confidence: Medium. The core architecture and integration are in place, but the final outcome depends on completing robust evaluation, managing real-world URL extraction variability, and ensuring that explanations and uncertainty messaging remain reliable and user-appropriate. These risks are explicitly planned for in the remaining timeline.

Reader Comments:

Declaration: I confirm that the work reported above is my own and accurately reflects my project progress to date.

Student Signature: ______________________ Date: ____________________